



REPUBLIQUE DU SENEGAL



Un Peuple - Un But - une Foi

MINISTRE DE L'ENSEIGNEMENT SUPERIEUR
DE LA RECHERCHE, ET DE L'INNOVATION

RAPPORT:
PYTHON/APACHE2/CGI/MYSQL

Présenté par Mouhamadou Falilou Sarr

Sous la direction de Pr Samuel Ouya

Décembre 2023

OBJECTIFS

Ce rapport a pour objectif de consolider nos connaissances en matière de développement web et d'administration de serveurs en se focalisant sur plusieurs compétences clés. Nous serons amenés à réviser et appliquer les concepts fondamentaux liés au protocole HTTP, notamment les formats de requêtes et réponses, ainsi que les verbes tels que GET, POST, PUT, DELETE.... De plus, nous renforcerons notre maîtrise des structures de données en Python, notamment les listes, tuples et dictionnaires, en apprenant à les manipuler et à les parcourir efficacement. Le TP abordera également les aspects de sécurité avec la génération de clés privées, la création de demandes de certificats SSL, et la signature de certificats. La configuration d'Apache2, en mettant l'accent sur les paramètres relatifs à la clé privée et au certificat x509, sera également explorée. En parallèle, nous réviserons les procédures d'administration d'une base de données MySQL, depuis la création d'un compte jusqu'à l'attribution de droits spécifiques sur une base de données. Enfin, nous développerons nos compétences d'exploration des modules Python en utilisant la fonction `dir()` dans l'interpréteur Python3 pour examiner les méthodes disponibles dans un module donné. L'ensemble de ces objectifs vise à nous fournir une expérience pratique et complète dans ces domaines cruciaux pour le développement web et l'administration de serveurs.

INTRODUCTION

Le présent rapport vise à renforcer nos compétences dans plusieurs domaines cruciaux pour le développement web et la gestion de serveurs. Pour tirer pleinement parti de cette série d'exercices, il est impératif de posséder des connaissances préalables en matière de protocole HTTP, de manipulation de données en Python, de sécurité des communications via SSL/TLS, de configuration d'Apache2, et d'administration de bases de données MySQL.

PARTIE 1 : redirection http vers https

Cette application a pour objectif de mettre en place une redirection automatique du protocole HTTP vers HTTPS sur notre site, afin d'assurer la sécurité des utilisateurs qui y accèdent.



Vérifions si apache2 est installé dans notre machine avec la commande : **apt policy apache2**

```
falilou@fs-ubuntu:~$ apt policy apache2
apache2:
  Installé : 2.4.41-4ubuntu3.15
  Candidat : 2.4.41-4ubuntu3.15
  Table de version :
  *** 2.4.41-4ubuntu3.15 500
      500 http://sn.archive.ubuntu.com/ubuntu focal-updates/main amd64 Package
s
      500 http://security.ubuntu.com/ubuntu focal-security/main amd64 Packages
      100 /var/lib/dpkg/status
  2.4.41-4ubuntu3 500
      500 http://sn.archive.ubuntu.com/ubuntu focal/main amd64 Packages
falilou@fs-ubuntu:~$
```



Activons le module ssl avec la commande : **a2enmod ssl**

```
falilou@fs-ubuntu:~$ sudo -s
[sudo] Mot de passe de falilou :
root@fs-ubuntu:/home/falilou# a2enmod ssl
Considering dependency setenvif for ssl:
Module setenvif already enabled
Considering dependency mime for ssl:
Module mime already enabled
Considering dependency socache_shmcb for ssl:
Enabling module socache_shmcb.
Enabling module ssl.
See /usr/share/doc/apache2/README.Debian.gz on how to configure SSL and create s
elf-signed certificates.
To activate the new configuration, you need to run:
    systemctl restart apache2
root@fs-ubuntu:/home/falilou#
```



Redémarrons le service apache2 pour appliquer les modifications avec la commande : **systemctl restart apache2**

```
systemctl restart apache2
root@fs-ubuntu:/home/falilou# systemctl restart apache2
root@fs-ubuntu:/home/falilou#
```



Activons le module de réécriture d'url d'apache2 avec la commande : **a2enmod rewrite**

```
root@fs-ubuntu:/home/falilou# systemctl restart apache2
root@fs-ubuntu:/home/falilou# a2enmod rewrite
Enabling module rewrite.
To activate the new configuration, you need to run:
    systemctl restart apache2
root@fs-ubuntu:/home/falilou#
```



Redémarrons de nouveau le service apache2 pour appliquer les nouvelles modifications avec la commande : **systemctl restart apache2**

```
systemctl restart apache2
root@fs-ubuntu:/home/falilou# systemctl restart apache2
root@fs-ubuntu:/home/falilou#
```



Créons et configurons le fichier de config du site mysite.conf dans **/etc/apache2/sites-available :**

```
GNU nano 4.8                               mysite.conf
<VirtualHost *:80>
    ServerName www.mfs.sn
    Redirect permanent / https://www.mfs.sn/
    ServerAdmin falilou@mfs.sn
    DocumentRoot /var/www/html/doc_mysite
</VirtualHost>
<VirtualHost *:443>
    ServerName www.mfs.sn
    ServerAdmin falilou@mfs.sn
    DocumentRoot /var/www/html/doc_mysite
    SSLEngine On
    SSLCertificateFile /etc/apache2/certificate/cert.pem
    SSLCertificateKeyFile /etc/apache2/certificate/key.pem
</VirtualHost>
```


Vérifions si openssl est installé dans notre machine :

```
root@fs-ubuntu:/# apt policy openssl
openssl:
  Installé : 1.1.1f-1ubuntu2.20
  Candidat : 1.1.1f-1ubuntu2.20
  Table de version :
 *** 1.1.1f-1ubuntu2.20 500
      500 http://sn.archive.ubuntu.com/ubuntu focal-updates/main amd64 Packages
      500 http://security.ubuntu.com/ubuntu focal-security/main amd64 Packages
      100 /var/lib/dpkg/status
 1.1.1f-1ubuntu2 500
      500 http://sn.archive.ubuntu.com/ubuntu focal/main amd64 Packages
root@fs-ubuntu:/#
```

Créons le dossier **certificate** dans /etc/apache2 :

```
root@fs-ubuntu:/etc/apache2# mkdir certificate
root@fs-ubuntu:/etc/apache2# cd certificate
root@fs-ubuntu:/etc/apache2/certificate#
```

Générons la clé privée avec la commande : **openssl genrsa -out key.pem**

```
root@fs-ubuntu:/etc/apache2/certificate# openssl genrsa -out key.pem
Generating RSA private key, 2048 bit long modulus (2 primes)
.....+++++
.....
e is 65537 (0x010001)
root@fs-ubuntu:/etc/apache2/certificate#
```

Générons la demande avec la commande : **openssl req -new -key key.pem -out cert.csr**

```
root@fs-ubuntu:/etc/apache2/certificate# openssl req -new -key key.pem -out cert.csr
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [AU]:SN
State or Province Name (full name) [Some-State]:SENEGAL
Locality Name (eg, city) []:THIES
Organization Name (eg, company) [Internet Widgits Pty Ltd]:MFS
Organizational Unit Name (eg, section) []:mfs
Common Name (e.g. server FQDN or YOUR name) []:mfs
Email Address []:

Please enter the following 'extra' attributes
to be sent with your certificate request
A challenge password []:passer
An optional company name []:
root@fs-ubuntu:/etc/apache2/certificate#
```



Signons le certificat avec la commande : **openssl x509 -signkey key.pem -in cert.csr -req -days 365 -out cert.pem**

```
root@fs-ubuntu:/etc/apache2/certificate# openssl x509 -signkey key.pem -in cert.csr -req -days 365 -out cert.pem
Signature ok
subject=C = SN, ST = SENEGAL, L = THIES, O = MFS, OU = mfs, CN = mfs
Getting Private key
root@fs-ubuntu:/etc/apache2/certificate#
```



Activons notre fichier de configuration avec la commande : **a2ensite mysite.conf**

```
root@fs-ubuntu:/etc/apache2/sites-available# a2ensite mysite.conf
Enabling site mysite.
To activate the new configuration, you need to run:
    systemctl reload apache2
root@fs-ubuntu:/etc/apache2/sites-available# systemctl reload apache2
root@fs-ubuntu:/etc/apache2/sites-available#
```



Ajoutons notre serveur dans /etc/hosts :

```
GNU nano 4.8 /etc/hosts
127.0.0.1    localhost    api.falilou.sn  www.mfs.sn
127.0.1.1    UBUNTU.falilou.virtualbox.org  UBUNTU

# The following lines are desirable for IPv6 capable hosts
::1         ip6-localhost ip6-loopback
fe00::0     ip6-localnet
ff00::0     ip6-mcastprefix
ff02::1     ip6-allnodes
ff02::2     ip6-allrouters
```



Allons dans le navigateur, tapons : **http://www.mfs.sn** et voir si on sera redirigé vers <https://www.mfs.sn> :



Attention : risque probable de sécurité

Firefox a détecté une menace de sécurité potentielle et n'a pas poursuivi vers www.mfs.sn. Si vous accédez à ce site, des attaquants pourraient dérober des informations comme vos mots de passe, e-mails, ou données de carte bancaire.

[En savoir plus...](#)

[Retour \(recommandé\)](#)

[Avancé...](#)

www.mfs.sn utilise un certificat de sécurité invalide.

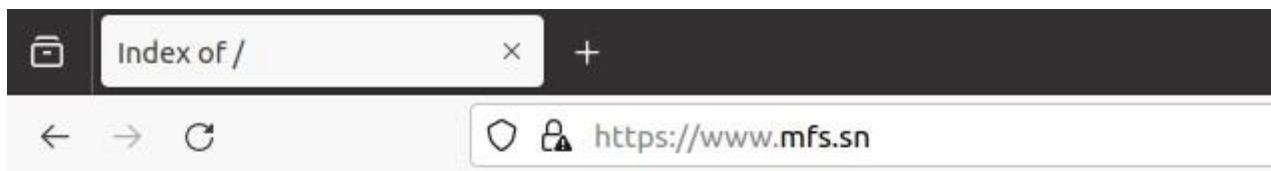
Le certificat n'est pas sûr car il est auto-signé.

Code d'erreur : [MOZILLA_PKIX_ERROR_SELF_SIGNED_CERT](#)

[Afficher le certificat](#)

[Retour \(recommandé\)](#)

[Accepter le risque et poursuivre](#)



Index of /

[Name](#) [Last modified](#) [Size](#) [Description](#)

Apache/2.4.41 (Ubuntu) Server at www.mfs.sn Port 443

PARTIE II : apache2 et cgi python

Cette application a pour objectif de savoir configurer Apache pour exécuter des scripts Python et ainsi mettre en place une application web fonctionnelle.



Activer le module **cgid** d'apache et redémarrons apache2 :

```
root@fs-ubuntu:/# a2enmod cgid
Module cgid already enabled
root@fs-ubuntu:/# systemctl restart apache2
root@fs-ubuntu:/#
```



Créons notre premier cgi test.py :

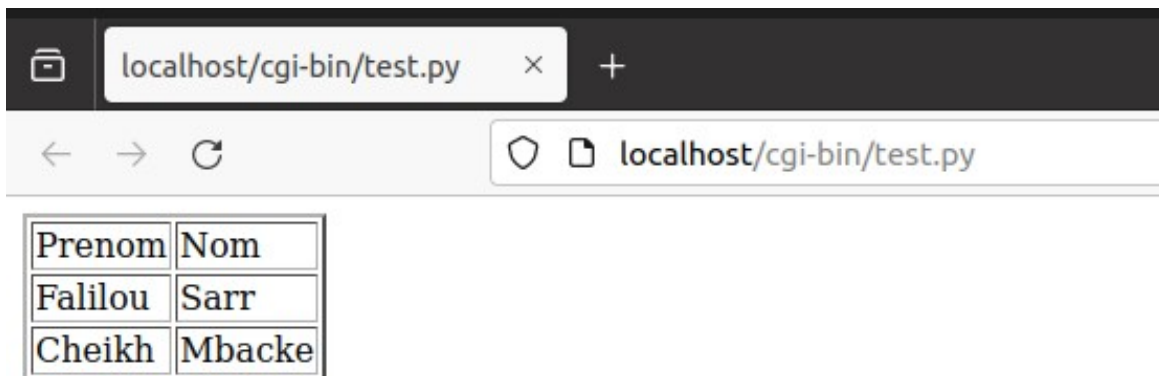
```
GNU nano 4.8 test.py
#!/usr/bin/python3
print('Content-Type: text/html')
print('')
chaîne = "<html><body> <table border='2px'><tr><td>Prenom</td><td>Nom</td></tr><tr><td>Famille</td><td>Sarr</td></tr><tr><td>Cheikh</td><td>Mbacke</td></tr></table></body></html>"
print(chaîne)
```

test.py

```
root@fs-ubuntu:/usr/lib/cgi-bin# nano test.py
root@fs-ubuntu:/usr/lib/cgi-bin# chmod 755 test.py
root@fs-ubuntu:/usr/lib/cgi-bin#
```

Testons notre programme python en tapant dans le navigateur :

http://localhost/cgi-bin/test.py



Prenom	Nom
Falilou	Sarr
Cheikh	Mbacke

PARTIE III: utilisation de l'environnement python et gestion de formulaire dans un script cgi python avec apache2

L'objectif de cette application est de maîtriser l'utilisation de l'environnement Python en conjonction avec Apache2 pour la gestion de formulaires dans un script CGI Python. Cela implique de comprendre comment intégrer efficacement des formulaires dans des scripts Python, de les traiter de manière appropriée, et de les déployer avec succès sous Apache2 pour créer des applications web interactives et fonctionnelles.

Afin de créer un environnement virtuel python nous allons installer **python3.8-venv** :

venv :

```
root@fs-ubuntu:/usr/lib/cgi-bin# apt install python3.8-venv
Lecture des listes de paquets... Fait
Construction de l'arbre des dépendances
Lecture des informations d'état... Fait
Les paquets supplémentaires suivants seront installés :
  libpython3.8 libpython3.8-dev libpython3.8-minimal libpython3.8-stdlib python3.8 python3.8-dev python3.8-minimal
Paquets suggérés :
  python3.8-doc binfmt-support
Les NOUVEAUX paquets suivants seront installés :
  python3.8-venv
Les paquets suivants seront mis à jour :
  libpython3.8 libpython3.8-dev libpython3.8-minimal libpython3.8-stdlib python3.8 python3.8-dev python3.8-minimal
7 mis à jour, 1 nouvellement installés, 0 à enlever et 76 non mis à jour.
Il est nécessaire de prendre 5.452 o/10,8 Mo dans les archives.
Après cette opération, 7.168 o d'espace disque seront libérés.
souhaitez-vous continuer ? [O/n]
97% [Connexion à sn.archive.ubuntu.com]
```




Créons un environnement virtuel nommé "**apache2**" en utilisant le module venv de Python 3 en tapant la commande : **python3 -m venv apache2**

```
root@fs-ubuntu:/# cd /usr/lib/cgi-bin
root@fs-ubuntu:/usr/lib/cgi-bin# python3 -m venv apache2
root@fs-ubuntu:/usr/lib/cgi-bin#
```



Vérifions si l'environnement virtuel et ses composants ont été créés :

```
root@fs-ubuntu:/usr/lib/cgi-bin# ls apache2/
bin  include  lib  lib64  pyvenv.cfg  share
root@fs-ubuntu:/usr/lib/cgi-bin#
```



Créons la base de données apache :

```
mysql> create database apache;
Query OK, 1 row affected (0,04 sec)

mysql> use apache
Database changed
mysql>
```



Créons la table etudiant :

```
mysql> create table client(id int primary key auto_increment,prenom varchar(30),nom varchar(30),code varchar(30),numcompte varchar(30),solde varchar(30));
Query OK, 0 rows affected (0,07 sec)

mysql> desc client;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| id    | int  | NO   | PRI | NULL    | auto_increment |
| prenom | varchar(30) | YES |     | NULL    |               |
| nom   | varchar(30) | YES |     | NULL    |               |
| code  | varchar(30) | YES |     | NULL    |               |
| numcompte | varchar(30) | YES |     | NULL    |               |
| solde | varchar(30) | YES |     | NULL    |               |
+-----+-----+-----+-----+-----+-----+
6 rows in set (0,03 sec)

mysql>
```

ACTIVITÉ 1 : Création d'un programme python qui fait des insertions de comptes bancaires dans la base de données



Créons notre formulaire d'ajout de comptes bancaires :

```
GNU nano 4.8                               formulaire.html
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0-alpha3/dist/css/bootstrap.min.css" rel="stylesheet" crossorigin="anonymous">
  <title>Formulaire</title>
</head>
<body class="d-flex justify-content-center align-items-center">
  <div class="container mx-auto" style="margin-top: 50px;">
    <form method="POST" action="/cgi-bin/crud.py">
      <legend><h1>Formulaire</h1></legend>
      <div class="form-group row">
        <label for="prenom" class="col-auto form-label">Prenom</label>
        <div class="mb-4 col-3">
          <input type="text" class="form-control" placeholder="Prenom" name="prenom">
        </div>
      </div>
      <div class="form-group row">
        <label for="nom" class="col-auto form-label">Nom</label>
        <div class="mb-4 col-3">
          <input type="text" class="form-control" placeholder="Nom" name="nom">
        </div>
      </div>
      <div class="form-group row">
        <label for="code" class="col-auto form-label">Code</label>
        <div class="mb-4 col-3">
          <input type="text" class="form-control" placeholder="Code" name="code">
        </div>
      </div>
      <div class="form-group row">
        <label for="numcompte" class="col-auto form-label">Numcompte</label>
        <div class="mb-4 col-3">
          <input type="text" class="form-control" placeholder="Numero de compte" name="numcompte" required>
        </div>
      </div>
      <div class="form-group row">
        <label for="solde" class="col-auto form-label">Solde</label>
        <div class="mb-4 col-3">
          <input type="text" class="form-control" placeholder="Solde" name="solde">
        </div>
      </div>
      <div class="mb-4 offset-2 col-1">
        <input type="submit" class="btn btn-success" value="Valider">
      </div>
    </form>
  </div>
</body>
</html>
```



Créons avec cgi et pymysql un programme python qui insert des comptes dans la base de données **apache** :

```
GNU nano 4.8                               crud.py
#!/usr/bin/python3
import cgi
import pymysql

def database():
    global conn, cursor
    conn = pymysql.connect(host="localhost", user='fallou', passwd='passer', database='apache')
    cursor = conn.cursor()

def insertion(prenom,nom,code,numcompte,solde):
    database()
    sql="insert into client(prenom,nom,code,numcompte,solde) values (%s, %s, %s, %s, %s)"
    val=(prenom,nom,code,numcompte,solde)
    cursor.execute(sql, val)
    conn.commit()

form = cgi.FieldStorage()
prenom = form.getvalue('prenom')
nom = form.getvalue('nom')
code = form.getvalue('code')
numcompte = form.getvalue('numcompte')
solde = form.getvalue('solde')

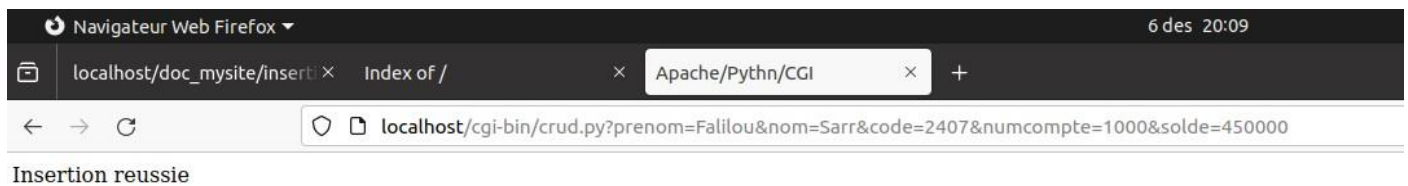
print ( "Content-type:text/html\r\n\r\n")
print ("<html>")
print ("<head>")
print ("<title>Apache/Python/CGI</title>")
print ("</head>")
print ("<body>")
print ("Insertion reussie")
print ("</body>")
print ("</html>")

insertion(prenom, nom, code, numcompte, solde)
#insertion("Fallou","Sarr","2407","1000","5400000")
```



Testons notre programme en utilisant un cgi via le client http suivant :

<http://localhost/cgi-bin/crud.py?prenom=Falilou&nom=Sarr&code=2407&numcompte=1000&solde=450000>



Vérifions dans notre base de donnée si le compte a été ajouté :

```
mysql> select * from client;
+----+-----+-----+-----+-----+-----+
| id | prenom | nom  | code | numcompte | solde |
+----+-----+-----+-----+-----+-----+
| 1  | Falilou | Sarr | 2407 | 1000      | 450000 |
+----+-----+-----+-----+-----+-----+
1 row in set (0,00 sec)

mysql>
```



Utilisons curl pour créer le compte de Papy avec la commande :

`curl -X GET "http://localhost/cgi-bin/crud.py?prenom=Papy&nom=Wade&code=1301&numcompte=1001&solde=670000"`

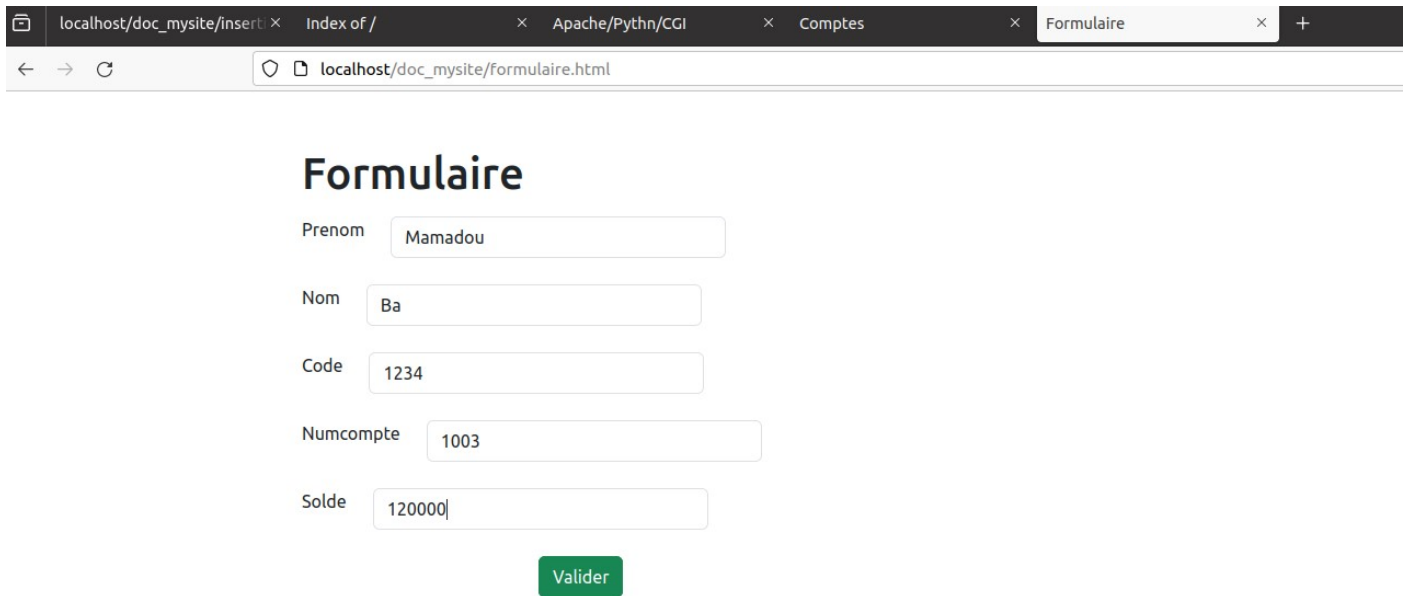
```
root@fs-ubuntu:/usr/lib/cgi-bin# curl -X GET "http://localhost/cgi-bin/crud.py?prenom=Papy&nom=Wade&code=1301&numcompte=1001&solde=670000"
<html>
<head>
<title>Apache/Python/CGI</title>
</head>
<body>
Insertion reussie
</body>
</html>
root@fs-ubuntu:/usr/lib/cgi-bin#
```



Vérifions dans notre base de donnée si le compte a été ajouté :

```
mysql> select * from client;
+----+-----+-----+-----+-----+-----+
| id | prenom | nom  | code | numcompte | solde |
+----+-----+-----+-----+-----+-----+
| 1  | Falilou | Sarr | 2407 | 1000      | 450000 |
| 2  | Papy    | Wade | 1301 | 1001      | 670000 |
+----+-----+-----+-----+-----+-----+
2 rows in set (0,00 sec)

mysql>
```



Formulaire

Prenom

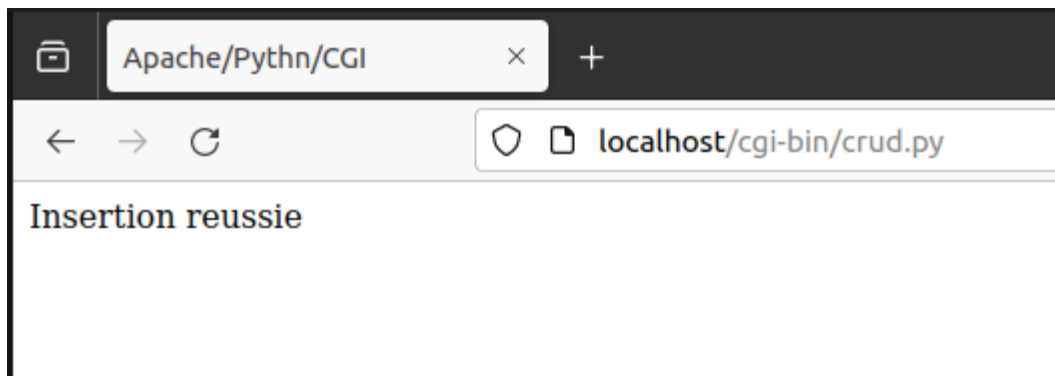
Nom

Code

Numcompte

Solde

Cliquons sur valider pour voir si ça marchera :



Insertion est réussie à présent allons dans la base de données pour vérifier :

```
mysql> select * from client;
+----+-----+-----+-----+-----+-----+
| id | prenom | nom  | code | numcompte | solde |
+----+-----+-----+-----+-----+-----+
| 1  | Falilou | Sarr | 2407 | 1000      | 450000 |
| 2  | Papy    | Wade | 1301 | 1001      | 670000 |
| 3  | Mamadou | Ba    | 1234 | 1003      | 120000 |
+----+-----+-----+-----+-----+-----+
3 rows in set (0,00 sec)

mysql>
```


ACTIVITÉ 2 : Création d'un programme python qui va se connecter à la base de données apache et récupère les informations



Créons un programme **lecture.py** qui va se connecter à la base de données apache, récupérer tous les enregistrements de la table client puis crée dynamiquement un tableau html pour afficher les enregistrements :

```
GNU nano 4.8 lecture.py
#!/usr/lib/cgi-bin/apache2/bin/python3
import pymysql

print("Content-Type: text/html")
print()
print("<html><head><title>Comptes</title></head>")
print("<body>")

connection = pymysql.connect(host="localhost", user='falilou', passwd='passer', database='apache')
cursor = connection.cursor()
cursor.execute("select * from client")

print('<table border="1px">')
print("<tr><td>Prenom</td><td>Nom</td><td>Code</td><td>Numcompte</td><td>Solde</td></tr>")

for row in cursor.fetchall():
    print(f"<tr><td>{row[1]}</td><td>{row[2]}</td><td>{row[3]}</td><td>{row[4]}</td><td>{row[5]}</td></tr>")
print("</table>")
print("</body></html>")

connection.close()
```



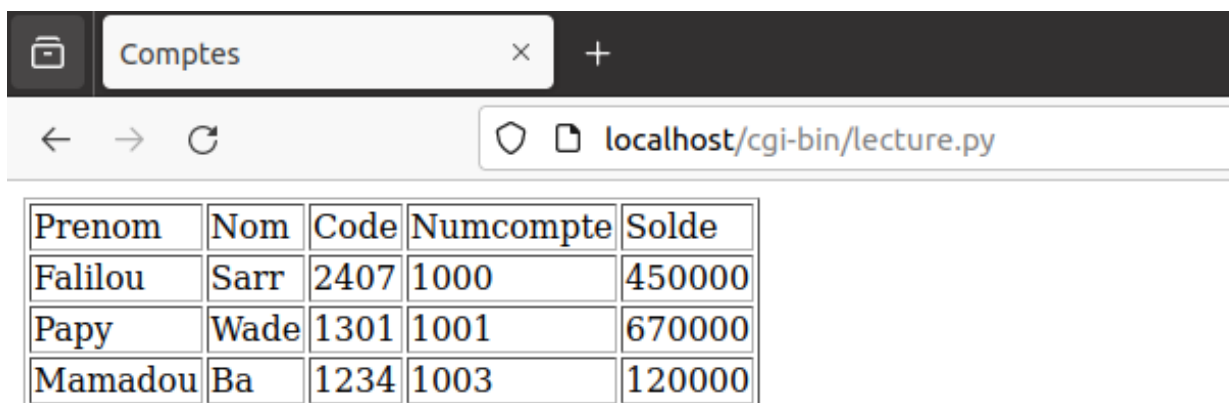
Rendons le script exécutable :

```
root@fs-ubuntu:/usr/lib/cgi-bin# chmod 755 lecture.py
root@fs-ubuntu:/usr/lib/cgi-bin#
```



Exécutons le script dans le navigateur en tapant :

<http://localhost/cgi-bin/lecture.py>



Prenom	Nom	Code	Numcompte	Solde
Falilou	Sarr	2407	1000	450000
Papy	Wade	1301	1001	670000
Mamadou	Ba	1234	1003	120000

ACTIVITÉ 3 : Installer le client http avancé postman et exécuter les 2 scripts python



Installons Postman à partir du snap avec la commande : **snap install postman**

```
root@fs-ubuntu:/usr/lib/cgi-bin# sudo snap install postman
Setup snap "postman" (234) security profiles for auto-connections
(postman-inc) installé
root@fs-ubuntu:/usr/lib/cgi-bin#
```



Lançons postman et inscrivons-nous à partir de l'interface graphique



Welcome to Postman ! Tell us a bit about yourself.

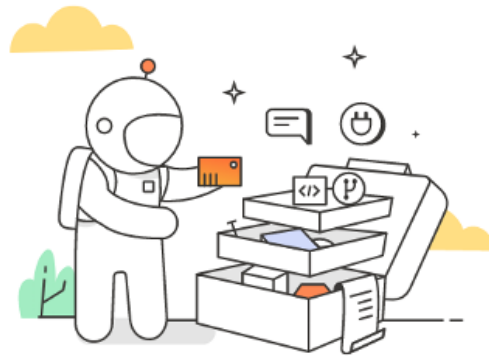
Your name

What is your role ?

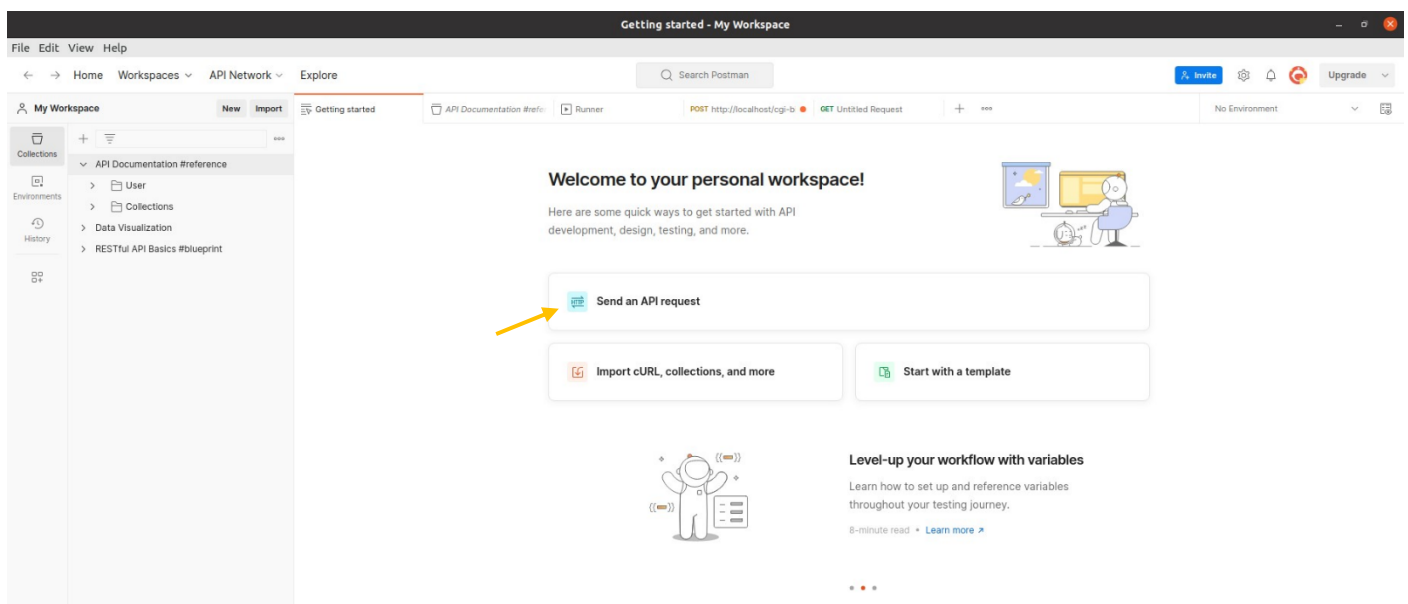
We will help you get the best out of Postman

Select your role

Continue



Après l'inscription, Accédons à l'accueil de postman et appuyons sur « **Send an API request** » :



1- Script d'insertion (crud.py)



On choisit la méthode **POST** et on ajoute le lien de notre script **crud.py** :

HTTP **POST** http://localhost/cgi-bin/crud.py

Params Authorization Headers (7) Body Pre-request Script Tests Settings

Query Params

Key	Value
Key	Value



Ensuite on clique sur '**Body**' puis sur '**x-www-form-urlencoded**' :

http://localhost/cgi-bin/crud.py Save

POST http://localhost/cgi-bin/crud.py Send

Params Authorization Headers (7) **Body** Pre-request Script Tests Settings

☐ none ☐ form-data ☒ x-www-form-urlencoded ☐ raw ☐ binary ☐ GraphQL

Key	Value	Description	...	Bulk Edit
Key	Value	Description		



Puis on clique sur '**Bulk Edit**' ensuite on fournit les données à insérer au postman et on clique sur '**send**' pour tout envoyer :

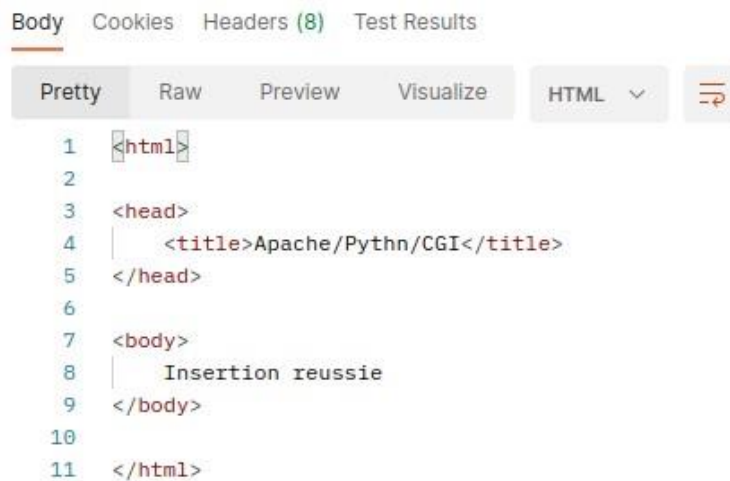
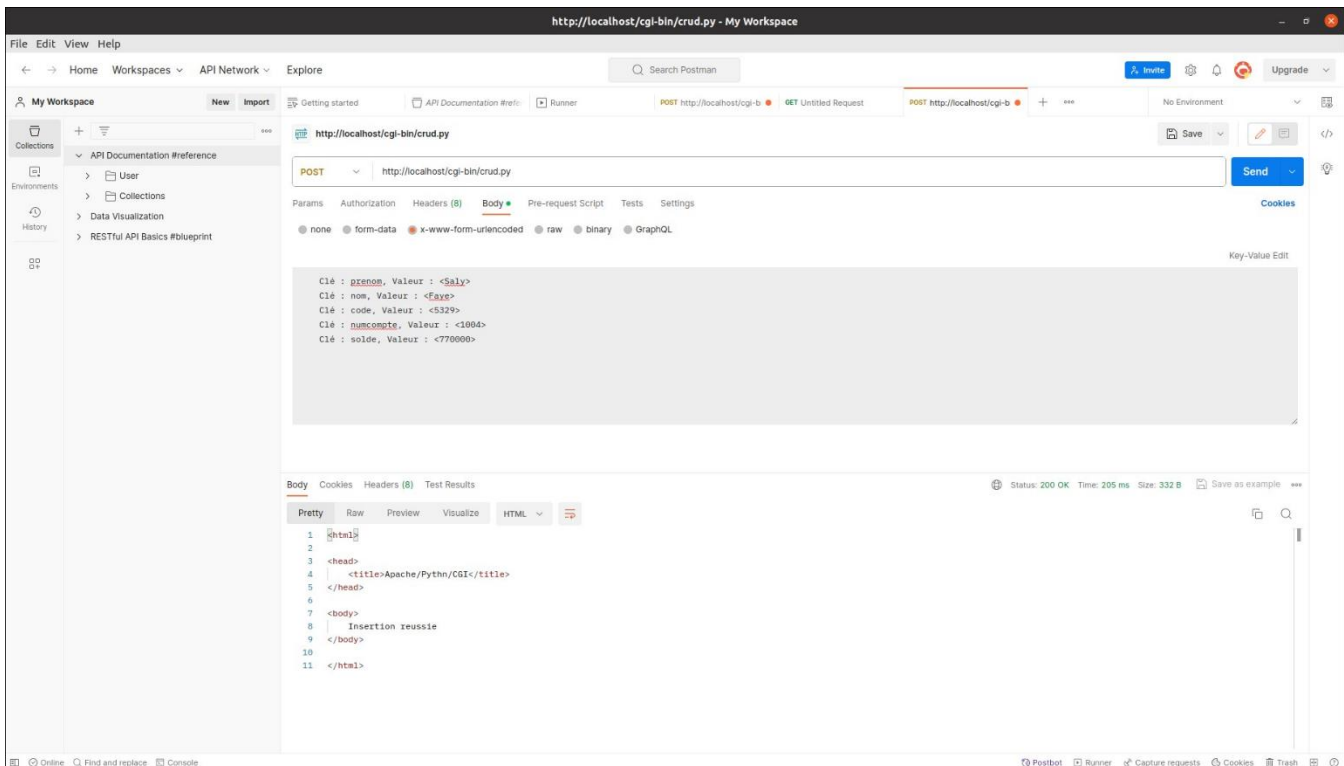
HTTP **POST** http://localhost/cgi-bin/crud.py

POST http://localhost/cgi-bin/crud.py

Params Authorization Headers (8) **Body** Pre-request Script Tests Settings

☐ none ☐ form-data ☒ x-www-form-urlencoded ☐ raw ☐ binary ☐ GraphQL

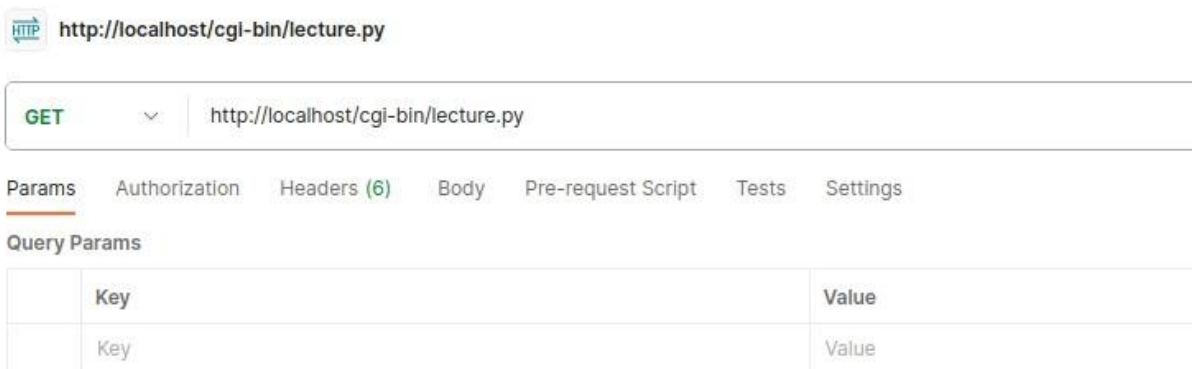
Clé='prenom', Valeur='Aliioune'
Clé='nom', Valeur='Ndiaye'
Clé='code', Valeur='7837'
Clé='numcompte', Valeur='1006'
Clé='solde', Valeur='555000'



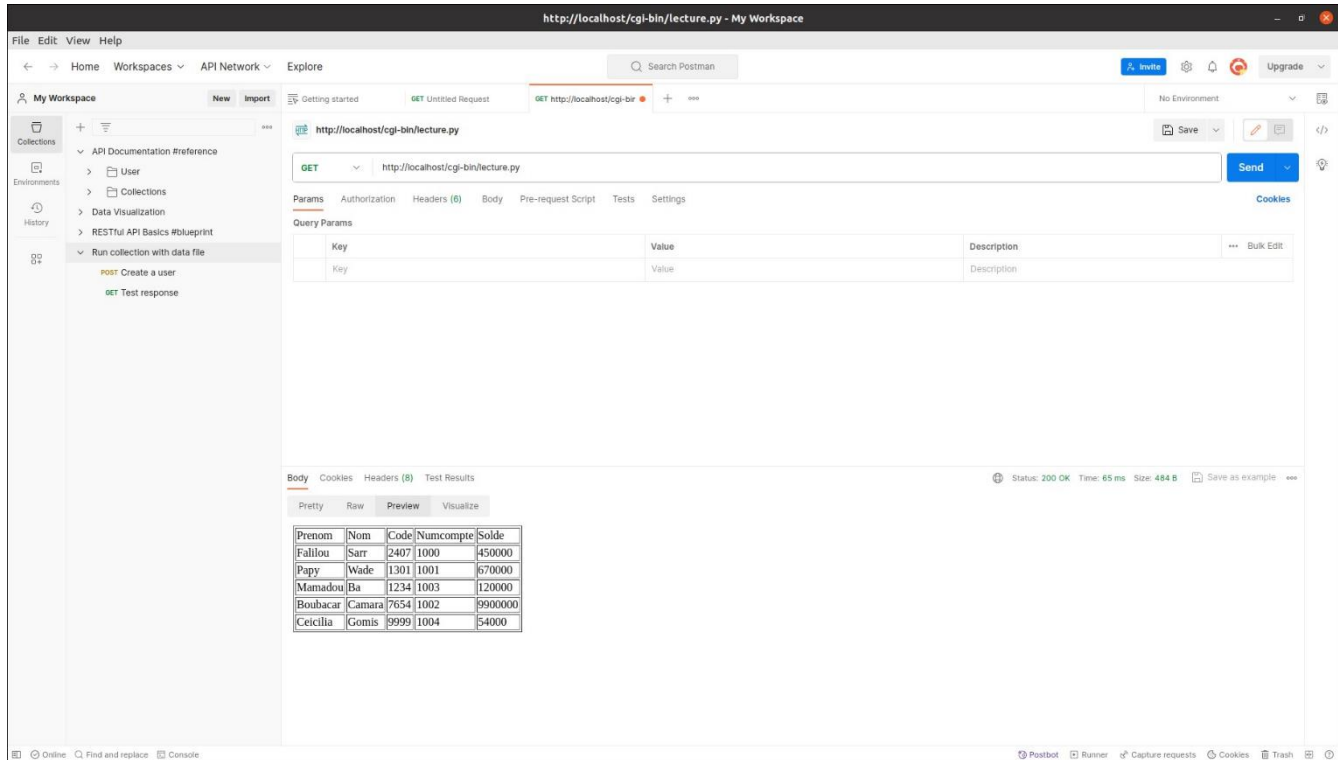
On voit que l'insertion est réussie !

2- Script de lecture (lecture.py)

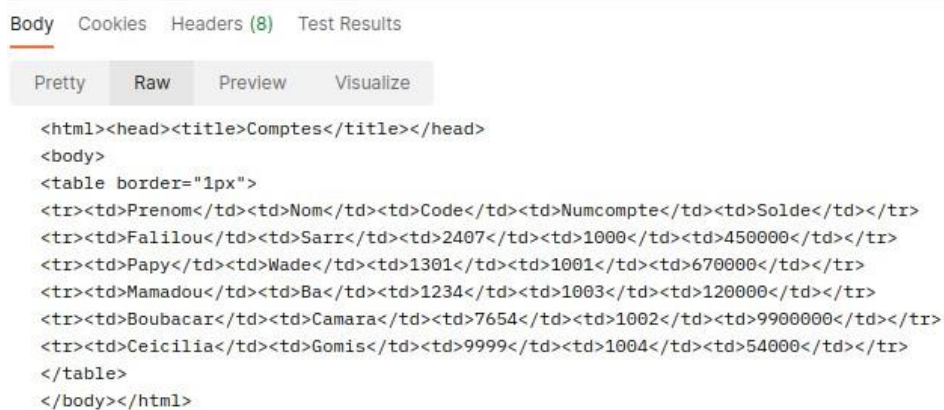
On choisit la méthode **GET** et on ajoute le lien de notre script **lecture.py** :



Contrairement au script d'insertion pour la lecture on a pas besoin de toucher body. On appuie directement sur **'send'** pour tout envoyer :



Affichage sous forme de ligne :



Affichage sous forme de tableau :

Prenom	Nom	Code	Numcompte	Solde
Falilou	Sarr	2407	1000	450000
Papy	Wade	1301	1001	670000
Mamadou	Ba	1234	1003	120000
Boubacar	Camara	7654	1002	9900000
Ceicilia	Gomis	9999	1004	54000

PARTIE IV: transformons les 2 derniers scripts en API REST

Dans cette partie, nous nous attaquons à une étape décisive pour moderniser notre solution logicielle : la transformation des deux derniers scripts en une API REST (Representational State Transfer). L'adoption d'une architecture orientée services via une API REST offre de nombreux avantages, notamment une meilleure extensibilité, une gestion simplifiée des fonctionnalités, et une intégration facilitée avec d'autres applications. Au cours de cette démarche, nous explorerons les étapes essentielles pour convertir nos scripts existants en un service web RESTful, ouvrant ainsi la voie à une utilisation plus efficace et flexible de notre application.

Une règle fondamentale d'api rest est que les données échangées entre les entités doivent être au format json.



Copions crud.py en creation.py avec la commande : **cp crud.py creation.py**

```
root@fs-ubuntu:/usr/lib/cgi-bin# cp crud.py creation.py
root@fs-ubuntu:/usr/lib/cgi-bin#
```



Entrons dans notre environnement virtuel avec la commande : **source apache2/bin/activate**

```
root@fs-ubuntu:/usr/lib/cgi-bin# source apache2/bin/activate
(apache2) root@fs-ubuntu:/usr/lib/cgi-bin#
```



Installons le module jsonify de prison en compte de format json avec la
Commande : **pip3 install jsonify**

```
(apache2) root@fs-ubuntu:/usr/lib/cgi-bin# pip3 install jsonify
Processing /root/.cache/pip/wheels/ee/c6/ef/643adfb90fefe0640cb6ba3b20995a2f50ccc2c57f55b190f/jsonify-0.5-py3-none-any.whl
Installing collected packages: jsonify
Successfully installed jsonify-0.5
(apache2) root@fs-ubuntu:/usr/lib/cgi-bin#
```

```

GNU nano 4.8                                                    creation.py
!apache2/bin/python3
import cgi,json
import pymysql

def database():
    global conn, cursor
    conn = pymysql.connect(host="localhost", user='falilou', passwd='passer', database='apache')
    cursor = conn.cursor()
def insertion(prenom,nom,code,numcompte,solde):
    database()
    sql="insert into client(prenom,nom,code,numcompte,solde) values (%s, %s, %s, %s, %s)"
    val=(prenom,nom,code,numcompte,solde)
    cursor.execute(sql, val)
    conn.commit()

form = cgi.FieldStorage()
prenom = form.getvalue('prenom')
nom = form.getvalue('nom')
code = form.getvalue('code')
numcompte = form.getvalue('numcompte')
solde = form.getvalue('solde')

print ( "Content-type:application/json\r\n\r\n")
print (json.dumps("Insertion reussie"))

insertion(prenom, nom, code, numcompte, solde)
#insertion("Falilou","Sarr","2407","1000","5400000")

```

Testons en ligne de commande avec la commande : **curl -X GET**

"http://localhost/cgi-bin/creation.py?prenom=Boubacar&nom=Camara&code=7654&numcompte=1002&solde=990000"

```

(apache2) root@fs-ubuntu:/usr/lib/cgi-bin# curl -X GET "http://localhost/cgi-bin/creation.py?prenom=Boubacar&nom=Camara&code=7654&numcompte=1002&solde=990000"
"Insertion reussie"
(apache2) root@fs-ubuntu:/usr/lib/cgi-bin#

```

Vérifions dans la base de données si Boubacar a été ajouté :

```

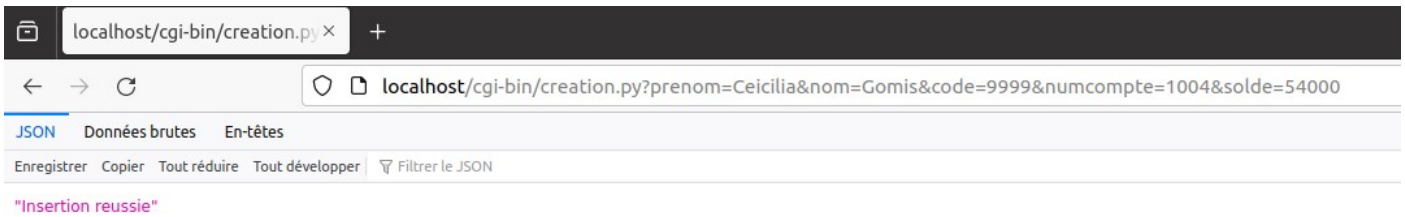
mysql> select * from client;
+----+-----+-----+-----+-----+-----+
| id | prenom | nom   | code | numcompte | solde |
+----+-----+-----+-----+-----+-----+
| 1  | Falilou | Sarr  | 2407 | 1000      | 450000 |
| 2  | Papy    | Wade | 1301 | 1001      | 670000 |
| 3  | Mamadou | Ba    | 1234 | 1003      | 120000 |
| 7  | Boubacar | Camara | 7654 | 1002      | 9900000 |
+----+-----+-----+-----+-----+-----+
4 rows in set (0,00 sec)

mysql>

```

Testons dans le navigateur en tapant : [http://localhost/cgi-](http://localhost/cgi-bin/creation.py?prenom=Cecilia&nom=Gomis&code=9999&numcompte=1004&solde=54000)

[bin/creation.py?prenom=Cecilia&nom=Gomis&code=9999&numcompte=1004&solde=54000](http://localhost/cgi-bin/creation.py?prenom=Cecilia&nom=Gomis&code=9999&numcompte=1004&solde=54000)



Vérifions dans la base de données si Cecilia a été ajouté :

```
mysql> select * from client;
+----+-----+-----+-----+-----+-----+
| id | prenom | nom   | code | numcompte | solde |
+----+-----+-----+-----+-----+-----+
| 1  | Falilou | Sarr  | 2407 | 1000      | 450000 |
| 2  | Papy    | Wade | 1301 | 1001      | 670000 |
| 3  | Mamadou | Ba    | 1234 | 1003      | 120000 |
| 7  | Boubacar | Camara | 7654 | 1002      | 9900000 |
| 8  | Ceicilia | Gomis | 9999 | 1004      | 54000  |
+----+-----+-----+-----+-----+-----+
5 rows in set (0,00 sec)

mysql>
```

Copions `lecture.py` en `lecture_json.py` :

```
(apache2) root@fs-ubuntu:/usr/lib/cgi-bin# cp lecture.py lecture_json.py
(apache2) root@fs-ubuntu:/usr/lib/cgi-bin#
```

Modifions `lecture_json.py` pour renvoyer juste les enregistrements de la table `client` sous forme de json :

```
GNU nano 4.8 lecture_json.py
#!/usr/lib/cgi-bin/apache2/bin/python3
import pymysql,json

print ("Content-Type: application/json")
print ()

connection = pymysql.connect(host="localhost", user='falilou', passwd='passer', database='apache')
cursor = connection.cursor()
cursor.execute("select * from client")

print (json.dumps(cursor.fetchall()))
connection.close()
```


Testons dans le navigateur en tapant la commande : **http://localhost/cgi-bin/lecture_json.py**

localhost/cgi-bin/lecture_json.py

JSON Données brutes En-têtes

Enregistrer Copier Tout réduire Tout développer Filtre le JSON

```

0:
  0: 1
  1: "Falilou"
  2: "Sarr"
  3: "2407"
  4: "1000"
  5: "450000"
1:
  0: 2
  1: "Papy"
  2: "Wade"
  3: "1301"
  4: "1001"
  5: "670000"
2:
  0: 3
  1: "Mamadou"
  2: "Ba"
  3: "1234"
  4: "1003"
  5: "120000"
3:
  0: 7
  1: "Boubacar"
  2: "Camara"
  3: "7654"
  4: "1002"
  5: "9900000"
4:
  0: 8
  1: "Ceicilia"
  2: "Gomis"
  3: "9999"
  4: "1004"
  5: "54000"
  
```

Installons le module **requests** permettant de faire de requêtes http :

```

(apache2) root@fs-ubuntu:/usr/lib/cgi-bin# pip3 install requests
WARNING: Retrying (Retry(total=4, connect=None, read=None, redirect=None, status=None)) after connection broken by '
0: Failed to establish a new connection: [Errno -3] Échec temporaire dans la résolution du nom)': /simple/requests
WARNING: Retrying (Retry(total=3, connect=None, read=None, redirect=None, status=None)) after connection broken by '
0: Failed to establish a new connection: [Errno -3] Échec temporaire dans la résolution du nom)': /simple/requests
WARNING: Retrying (Retry(total=2, connect=None, read=None, redirect=None, status=None)) after connection broken by '
0: Failed to establish a new connection: [Errno -3] Échec temporaire dans la résolution du nom)': /simple/requests
Collecting requests
  Downloading requests-2.31.0-py3-none-any.whl (62 kB)
    | 62 kB 305 kB/s
Collecting certifi<=2017.4.17
  Downloading certifi-2023.11.17-py3-none-any.whl (162 kB)
    | 162 kB 1.5 MB/s
Collecting idna<4,>=2.5
  Downloading idna-3.6-py3-none-any.whl (61 kB)
    | 61 kB 78 kB/s
Collecting urllib3<3,>=1.21.1
  Downloading urllib3-2.1.0-py3-none-any.whl (104 kB)
    | 104 kB 8.6 MB/s
Collecting charset-normalizer<4,>=2
  Downloading charset-normalizer-3.3.2-cp38-cp38-manylinux_2_17_x86_64_manylinux2014_x86_64.whl (141 kB)
    | 141 kB 6.3 MB/s
Installing collected packages: certifi, idna, urllib3, charset-normalizer, requests
Successfully installed certifi-2023.11.17 charset-normalizer-3.3.2 idna-3.6 requests-2.31.0 urllib3-2.1.0
(apache2) root@fs-ubuntu:/usr/lib/cgi-bin#
  
```




Créons **frontendlecturejson.py** qui sera une vue en python qui récupère les données via lecture_json.py les parcourt et affiche les résultats sous forme de ligne :

```
GNU nano 4.8
#!/usr/bin/env python3
import requests
import json

print("Content-type: application/json\r\n\r\n")

response = requests.get('http://localhost/cgi-bin/lecture_json.py')
print(response.json())
```



Rendons le fichier exécutable :

```
(apache2) root@fs-ubuntu:/usr/lib/cgi-bin# chmod 755 frontendlecturejson.py
(apache2) root@fs-ubuntu:/usr/lib/cgi-bin#
```



Testons dans le navigateur en tapant : **http://localhost/cgi-bin/frontendlecturejson.py**



Testons en ligne de commande avec curl : **curl -X "http://localhost/cgi-bin/frontendlecturejson.py"**

```
(apache2) root@fs-ubuntu:/usr/lib/cgi-bin# curl -X GET "http://localhost/cgi-bin/frontendlecturejson.py"
[[{"1", "Fallilou", "Sarr", "2407", "1000", "450000"}, {"2", "Papy", "Wade", "1301", "1001", "670000"}, {"3", "Mamadou", "Ba", "1234", "1003", "120000"}, {"7", "Boubacar", "Camara", "7654", "1002", "99000000"}, {"8", "Ceicilia", "Gomis", "9999", "1004", "54000"}]]
(apache2) root@fs-ubuntu:/usr/lib/cgi-bin#
```



Modifions le script pour que le résultat soit une liste qu'on va parcourir pour récupérer les enregistrements

```
GNU nano 4.8
#!/apache2/bin/python3
import requests,json
print( "Content-type:text/html\r\n\r\n")
response = requests.get('http://localhost/cgi-bin/lecturejson.py')
#print(response.json())
print(type(response.json()))
```



Testons en ligne de commande avec curl : **curl -X "http://localhost/cgi-bin/frontendlecturejson.py"**

```
(apache2) root@fs-ubuntu:/usr/lib/cgi-bin# curl -X GET "http://localhost/cgi-bin/frontendlecturejson.py"
<class 'list'>
(apache2) root@fs-ubuntu:/usr/lib/cgi-bin#
```



Modifions le code pour que les enregistrements soient en ligne par ligne

```
GNU nano 4.8
#!/apache2/bin/python3
import requests,json
print( "Content-type:text/html\r\n\r\n")
response = requests.get('http://localhost/cgi-bin/lecturejson.py')
liste1=response.json()
#print(type(response.json()))
for ligne in liste1:
print(ligne)
```



Testons avec curl en ligne de commande : **curl -X "http://localhost/cgi-bin/frontendlecturejson.py"**

```
(apache2) root@fs-ubuntu:/usr/lib/cgi-bin# curl -X GET "http://localhost/cgi-bin/frontendlecturejson.py"
[1, 'Falilou', 'Sarr', '2407', '1000', '450000']
[2, 'Papy', 'Wade', '1301', '1001', '670000']
[3, 'Mamadou', 'Ba', '1234', '1003', '120000']
[7, 'Boubacar', 'Camara', '7654', '1002', '9900000']
[8, 'Ceicilia', 'Gomis', '9999', '1004', '54000']
(apache2) root@fs-ubuntu:/usr/lib/cgi-bin#
```



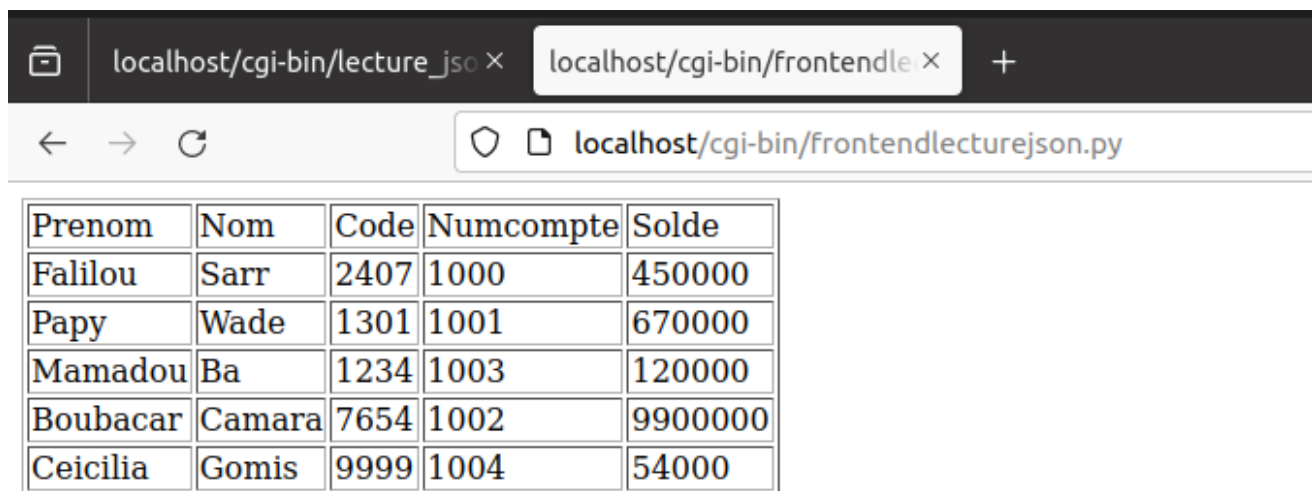
Modifions encore le code **frontendlecturejson.py** pour afficher les enregistrements sous forme de tableau :

```
GNU nano 4.8 frontendlecturejson.py
#!/apache2/bin/python3
import requests,json
print( "Content-type:text/html\r\n\r\n")

response = requests.get('http://localhost/cgi-bin/lecture_json.py')
liste1=response.json()

#print(type(response.json()))
print ('<table border="1px">')
print ("<tr><td>Prenom</td><td>Nom</td><td>Code</td><td>Numcompte</td><td>Solde</td></tr>")
for ligne in liste1:
    print (f"<tr><td>{ligne[1]}</td><td>{ligne[2]}</td><td>{ligne[3]}</td><td>{ligne[4]}</td><td>{ligne[5]}</td></tr>")
print ("</table>")
print ("</body></html>")
```

Testons dans le navigateur en tapant : **http://localhost/cgi-bin/frontendlecturejson.py**



The screenshot shows a web browser window with the address bar displaying 'localhost/cgi-bin/frontendlecturejson.py'. The page content is a table with 5 columns: Prenom, Nom, Code, Numcompte, and Solde. The table contains 6 rows of data.

Prenom	Nom	Code	Numcompte	Solde
Falilou	Sarr	2407	1000	450000
Papy	Wade	1301	1001	670000
Mamadou	Ba	1234	1003	120000
Boubacar	Camara	7654	1002	9900000
Ceicilia	Gomis	9999	1004	54000

PARTIE V: frontal apache2 via proxy et proxyreverse

Cette partie se concentre sur la sécurisation de notre application web en cachant sa véritable URL grâce à l'utilisation de deux composants clés : le proxy et le proxy reverse. Ces éléments agissent comme des intermédiaires entre les clients et les serveurs, renforçant ainsi la sécurité et préservant la confidentialité des ressources en ligne. Nous explorerons les principes de fonctionnement de ces mécanismes et discuterons de leur mise en œuvre pratique, mettant en lumière les avantages qu'ils apportent, tels que la défense contre les menaces potentielles et une gestion optimisée des requêtes.

Activons les 2 modules **proxy** et **proxy_http** :

```
root@fs-ubuntu:/usr/lib/cgi-bin# a2enmod proxy proxy_http
Enabling module proxy.
Considering dependency proxy for proxy_http:
Module proxy already enabled
Enabling module proxy_http.
To activate the new configuration, you need to run:
  systemctl restart apache2
root@fs-ubuntu:/usr/lib/cgi-bin#
```

Redémarrons le service apache2 pour appliquer les modifications :

```
root@fs-ubuntu:/usr/lib/cgi-bin# systemctl restart apache2
root@fs-ubuntu:/usr/lib/cgi-bin#
```

```
GNU nano 4.8 /etc/apache2/sites-available/proxyfrontend.conf
<VirtualHost *:80>
ServerName frontend.falilou.sn
ProxyPass / http://127.0.0.1/cgi-bin/frontendlecturejson.py/
ProxyPassReverse / http://127.0.0.1/cgi-bin/frontendlecturejson.py/
ProxyRequests Off
</VirtualHost>
```

Activons le site :

```
root@fs-ubuntu:/etc/apache2/sites-available# a2enmod proxyfrontend.conf
ERROR: Module proxyfrontend.conf does not exist!
root@fs-ubuntu:/etc/apache2/sites-available# a2ensite proxyfrontend.conf
Enabling site proxyfrontend.
To activate the new configuration, you need to run:
systemctl reload apache2
root@fs-ubuntu:/etc/apache2/sites-available#
```

Rechargeons le service apache2 pour appliquer les modifications :

```
root@fs-ubuntu:/etc/apache2/sites-available# systemctl reload apache2
root@fs-ubuntu:/etc/apache2/sites-available#
```

Ajoutons notre serveur dans /etc/hosts :

```
GNU nano 4.8 /etc/hosts
127.0.0.1 localhost api.falilou.sn www.mfs.sn frontend.falilou.sn
127.0.1.1 UBUNTU.falilou.virtualbox.org UBUNTU

# The following lines are desirable for IPv6 capable hosts
::1 ip6-localhost ip6-loopback
fe00::0 ip6-localnet
ff00::0 ip6-mcastprefix
ff02::1 ip6-allnodes
ff02::2 ip6-allrouters
```

Testons le site dans le navigateur en tapant **frontend.falilou.sn** :

localhost/cgi-bin/lecture_jsc × localhost/cgi-bin/frontendle × frontend.falilou.sn/ × +

← → ↻
frontend.falilou.sn

Prenom	Nom	Code	Numcompte	Solde
Falilou	Sarr	2407	1000	450000
Papy	Wade	1301	1001	670000
Mamadou	Ba	1234	1003	120000
Boubacar	Camara	7654	1002	9900000
Ceicilia	Gomis	9999	1004	54000

La personne qui utilise notre frontend en passant par l'url frontend.falilou.sn ne sait pas dans quel langage nous avons développé notre frontend.

CONCLUSION

En conclusion de cette série d'activités, notre rapport détaille les étapes essentielles pour renforcer la sécurité, la fonctionnalité et l'accessibilité de notre application web. Dans la première partie, nous avons mis en place une redirection automatique du protocole HTTP vers HTTPS, garantissant ainsi un accès sécurisé pour les utilisateurs. En suivant des étapes méthodiques, nous avons configuré Apache, généré des certificats SSL, et testé avec succès la redirection.

La deuxième partie a traité de la configuration d'Apache pour exécuter des scripts Python via le module cgi. En créant un script CGI test.py, nous avons démontré la capacité d'Apache à traiter des scripts Python de manière efficace, ouvrant ainsi la voie à des applications web fonctionnelles.

La troisième partie s'est concentrée sur l'utilisation de l'environnement Python pour la gestion de formulaires dans un script CGI avec Apache2. Nous avons créé un environnement virtuel, configuré une base de données, et développé des scripts Python pour l'insertion et la récupération de données via des formulaires web.

Dans la quatrième partie, nous avons transformé nos scripts Python en une API REST, adoptant une architecture orientée services. En respectant les principes REST, nous avons adapté nos scripts pour répondre aux requêtes HTTP et renvoyer les résultats au format JSON, améliorant ainsi la flexibilité et l'interopérabilité de notre application.

Enfin, la cinquième partie a abordé la sécurisation de notre application en cachant sa véritable URL grâce à l'utilisation de proxies et de proxy reverse. En activant les modules proxy et proxy_http, nous avons créé un site virtuel et configuré un proxy reverse, renforçant ainsi la confidentialité des ressources en ligne.

Dans l'ensemble, ces activités ont contribué à moderniser notre solution logicielle, combinant des éléments de sécurité, de fonctionnalité avancée, et d'intégration de services web. En adoptant ces meilleures pratiques, nos applications seront mieux préparées pour répondre aux défis actuels et futurs du développement web.